Independent Study Course (NST3901)

Theoretical Foundations for 3D Reconstruction of Sports Games with Limited Visual Data Settings

14.11.2025

Nguyen Hoang The Kiet
NUS College, National University of Singapore

Contents

[1] Adopted from the ISC application form

List of Figures

[2] Adopted from the ISC application form

1. Introduction

1.1. Motivation[2]

The Video Assistant Referee (VAR) system was introduced at the 2018 FIFA World Cup in Russia, after which it became popularised in other European domestic and continental leagues. Originally a video review system for referees in case of critical fouls (e.g., offsides leading to a goal, offences in the penalty box, etc.), the technology has been equipped with major upgrades, such as the semi-automated offside technology (SAOT) used in the latest edition of the FIFA World Cup in Qatar [2].

Figure 1: Semi-automated offside technology used in Italy's Serie A.

Lower-tier tournaments such as the 2024 ASEAN Cup (formerly known as AFF Cup) and domestic leagues in Southeast Asia are also improving transparency in refereeing decisions by implementing low-budget versions of the standard Video Assistant Referee (VAR), such as VAR Lite or centralised VAR. However, these budget systems quickly showed vulnerability in operation, especially in high-stakes situations. For instance, in the two legs of the Thailand vs. Philippines match-off in the 2nd semifinal of the ASEAN Cup:

- In the first leg [3], VAR did not participate in a potential offside offence by the Filipino player Alex Munis, which then led to the Philippines' first goal in their historic 2-1 win against Thailand. Speculations from social media then showed that VAR seemed not to have worked at all due to the latency between the field and the centralised VAR station in Singapore.
- In the second leg [4], VAR also did not intervene when the Thai player Seksan Ratree allegedly dribbled the ball crossing the goal line, leading to Peradol's opening goal for Thailand that would win them the leg and the semi-final. The replay was indecisive and caused many controversies afterwards.

Figure 2: Two of the controversial incidents in the Thailand v Philippines ASEAN Cup semi-final legs.

In light of the mentioned shortcomings, this project aims to build an end-to-end solution for reconstructing game events in 3D under limited data conditions, with potential applications in refereeing, analytics and broadcasting.

1.2. Scope of the Independent Study Course project

The first stage of the project will be a survey of foundational methods in the field registration stage. The aim is to reconstruct a model of a football field by locating all lines with its respected position on the field. Any image of a football can be uniquely described by a homography transformation from a template football field. This transformation will be the input for the *Camera calibration* task through calibration algorithms such as Zhang's [5] or Bradski's [6], which will be part of the second milestone of the project.

1.3. The suggested pipeline

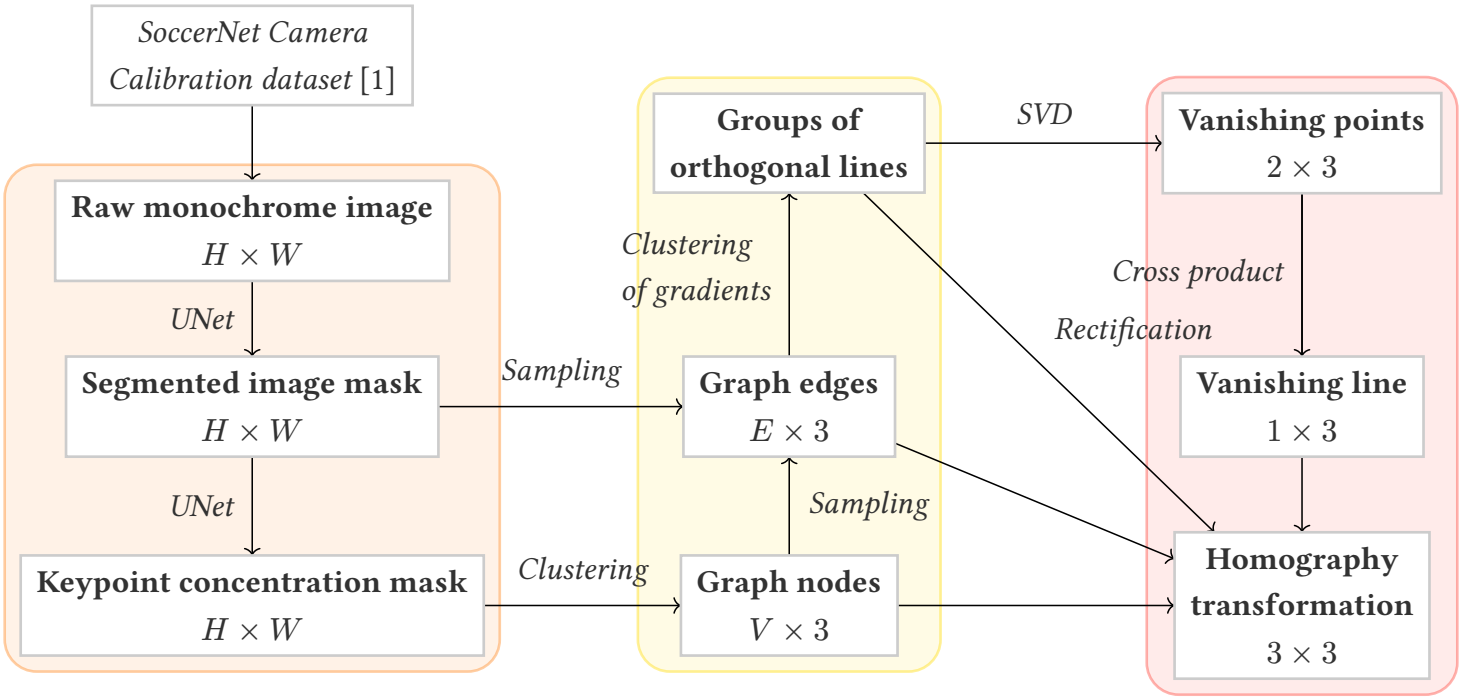


Figure 3: Summary of the suggested automated pipeline for field registration

We propose the above automated pipeline for the field registration problem, which can be divided into three smaller subtasks with its respective domain of knowledge:

- **Feature masks generation**. Using a generative image model (for instance, U-Net [7]), we seperate the important features from the raw image of the field, i.e. the field lines and keypoints, in the form of a gray-scale image mask.

- **Graph creation**. From the image masks, we detect the keypoints in coordinates form and construct the edges between them. Due to the potential noise of the masks, however, the challenge is to make the algorithms robust through techniques such as gamma correction and sampling. Finally, exploiting the orthogonal structure of the field lines, we classify each field line as either 'vertical' or 'horizontal.' This additional information will assist the following step.

- **Homography reconstruction**. From the constructed graph and the coordinates of each keypoint, we shall reconstruct a transformation from the world view (described by a template field) to the camera view (given by the image). Such a transformation from a template, planar field to the transformed field is called a homography, the process of recovering which is well described in textbooks such as Hartley and Zisserman [8]. However, as we do not have a keypoint-keypoint mapping, such correspondence must be found through an exhaustive search.

2. Feature masks generation via generative models

2.1. Generative models as feature generators

The first step for field registration is to extract the field lines and the keypoints on the field, masking out irrelevant portions of the image such as the audience, broadcasting displays, etc. Notice that these objects for detection are generally not suitable for a object detection framework such as YOLO or CNN-bsaed ones. Such traditional frameworks rely on the object being trustfully detected via a bounding box [9], which seems not suitable for geometric objects such as lines. Therefore, a generative model approach is adopted: we extract all pixels that contribute to the field lines, after which we use classical algorithms and unsupervised learning methods to extract the coordinates of the keypoints and the line segments.

We use the U-Net architecture as the generative model for its strength on localisation tasks and its ability to learn fast even in low data conditions [7].

2.2. Problem formulation and the choice of a loss function

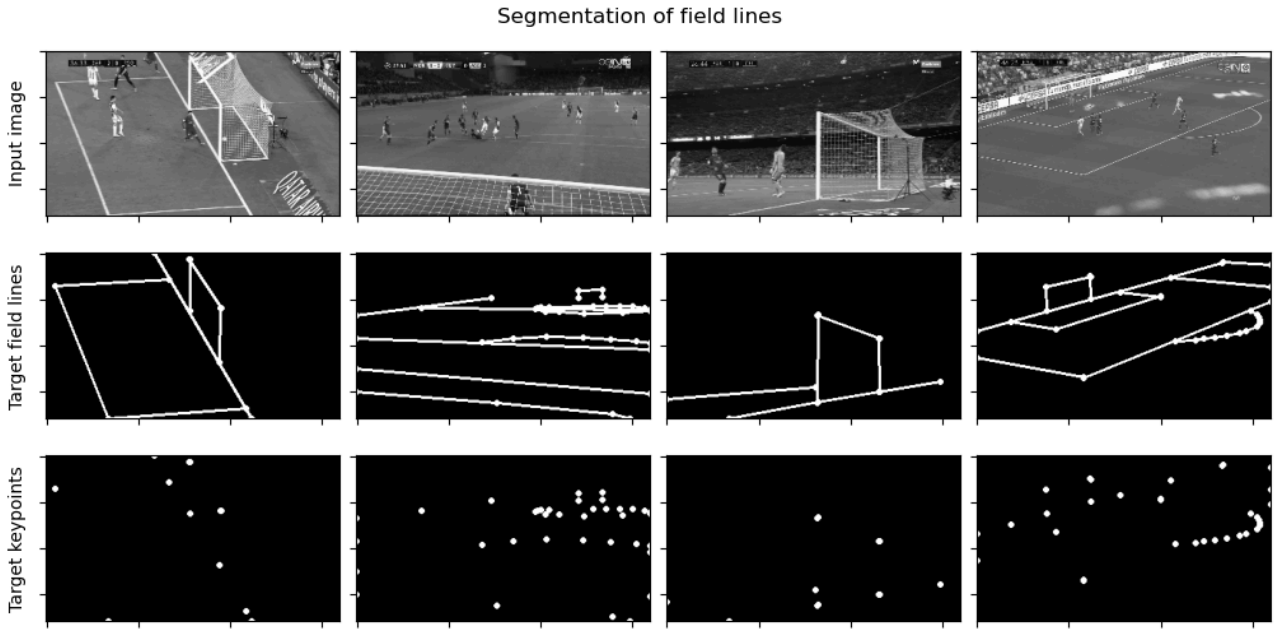


Figure 4: A generative model based pipeline:
Input image $(H \times W) \longrightarrow$ Target field lines mask $(H \times W) \longrightarrow$ Target keypoints $(H \times W)$

We formulate the problems as follows: Given a gray-scaled input image I of size $H \times W$, we want to train two models $\mathcal{U}_{\text{seg}}$ and $\mathcal{U}_{\text{kp}}$ such that $\mathcal{U}_{\text{seg}}(I)$ intensifies the white pixels representing the field lines and $\mathcal{U}_{\text{kp}}\big(\mathcal{U}_{\text{seg}}(I)\big)$ retains only the pixels around the keypoints (intersection between different field lines) of the image. The loss is defined as

$$\mathcal{L}_{\text{seg}} = \mathcal{L}\big(I, \mathcal{U}_{\text{seg}}(I)\big)$$

,

$$\mathcal{L}_{\text{kp}} = \mathcal{L}\big(I, \mathcal{U}_{\text{kp}}\big(\mathcal{U}_{\text{seg}}(I)\big)\big)$$

where $\mathcal{L}$ is a loss function, treating the image masks as a column vector of pixel intensities. The training process follows by training and validating $\mathcal{U}_{\text{seg}}$ first, then training and validating $\mathcal{U}_{\text{kp}}$, considering $\mathcal{U}_{\text{seg}}$ as fixed.

For the choice of the loss function, we surveyed the following losses from [10]:

- **Cross Binary Entropy Loss**

$$\mathcal{L}_{\text{BCE}}(\vec{\boldsymbol{x}}, \vec{\boldsymbol{y}}) = -\sum_i \log(x_i y_i) = -\log(\vec{\boldsymbol{x}}) \cdot \log(\vec{\boldsymbol{y}})$$

- **Intersection-over-Union (IOU) Loss**

$$\mathcal{L}_{\text{IOU}}(\vec{\boldsymbol{x}}, \vec{\boldsymbol{y}}) = \frac{|X \cap Y|}{|X \cup Y|} = \frac{\vec{\boldsymbol{x}} \cdot \vec{\boldsymbol{y}}}{1 - \left(\vec{\mathbb{1}} - \vec{\boldsymbol{x}}\right)\left(\vec{\mathbb{1}} - \vec{\boldsymbol{y}}\right)}$$

- **Dice Loss** (for binary classes)

$$\mathcal{L}_{\text{Dice}}(\vec{\boldsymbol{x}}, \vec{\boldsymbol{y}}) = \frac{2|X \cap Y|}{|X| + |Y|} = 2 \cdot \frac{\vec{\boldsymbol{x}} \cdot \vec{\boldsymbol{y}}}{\vec{\boldsymbol{x}} \cdot \vec{\mathbb{1}} + \vec{\boldsymbol{y}} \cdot \vec{\mathbb{1}}}$$

We settled with the Dice Loss due to its interpretability (the loss value proportional to the count of mismatched pixels) and ease of computation (only involves the intersection, compared to the IOU which involves both the intersection and union operator).

Detailed training results are presented under Section 5.2.

3. Detecting keypoints and graph edges via clustering

3.1. Detecting keypoints through clustering methods

From the keypoint concentrated mask, we extract all coordinates $\left(x^{(i)}, y^{(i)}\right)$ whose pixel intensity level exceeds a threshold level T ($T \approx 0.8$). We shall thus obtain a list of "white" pixels $\boldsymbol{P} = \left[\vec{\boldsymbol{p}}_1 \;\; \vec{\boldsymbol{p}}_2 \;\; \cdots \;\; \vec{\boldsymbol{p}}_m\right]$. This list is yet the desired keypoints list, instead we observe from the mask that each keypoint is associated with a "cluster" of "white" pixels, which means that a keypoint is the average of some "white" points $\vec{\boldsymbol{p}}_i$ within its clutser. A similar approach can be seen in [11].

The above observation motivates us for a keypoint positioning algorithm via clustering methods as follows:

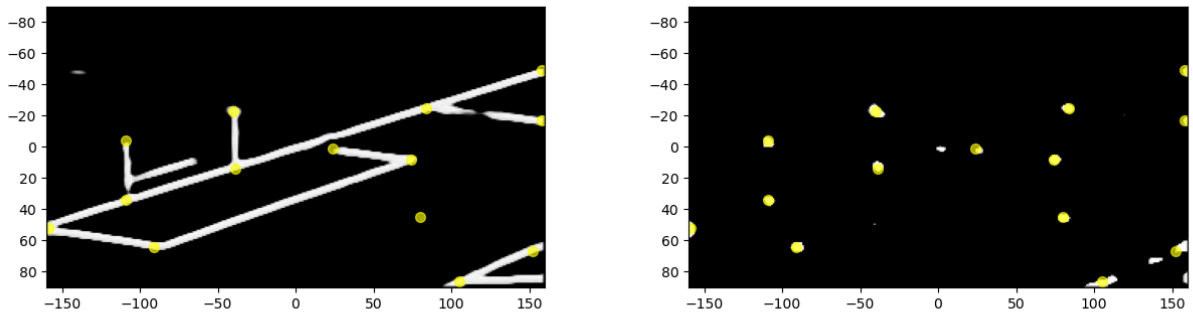


Figure 5: Outline mask (left) and keypoint concentrated mask (right)

Keypoints detection via clustering of "white" pixels

- *Input.*
  - Keypoint concentrated mask I_K of size $H \times W$.
  - Threshold $T \approx 0.8$
- *Output.* List of keypoints

$$P = \begin{bmatrix} x^{(1)} & x^{(2)} & \cdots & x^{(n)} \\ y^{(1)} & y^{(2)} & \cdots & y^{(n)} \end{bmatrix}$$

- *Hyperparameters.* A clustering algorithm $\mathcal{C}$.

Step 1. From the keypoint concentrated mask, extract all coordinates $\left(x^{(i)}, y^{(i)}\right)$ whose pixel intensity level exceeds a threshold level T, thus obtaining a list of "white" pixels $\boldsymbol{Q} = \left[\vec{\boldsymbol{q}}^{(1)} \;\; \vec{\boldsymbol{q}}^{(2)} \;\; \cdots \;\; \vec{\boldsymbol{q}}^{(m)}\right]$.

Step 2. Run the clustering algorithm $\mathcal{C}$ on $\boldsymbol{Q}$, extract all **cluster centroids** $\boldsymbol{P} = [\vec{\boldsymbol{p}}^{(1)}\ \vec{\boldsymbol{p}}^{(2)}\ \cdots\ \vec{\boldsymbol{p}}^{(m)}]$.

3.2. Detecting edges through sampling

For each pair of keypoints, we want to establish a graph of connecting edges between the keypoints if there is a line segment connecting them on the image. As the segmented mask has seperated the filed lines with the rest of the image, yet still noisy, we need a robust algorithm to detect an edge between any keypoints pair.

Our idea is to sample as many points as possible lying between the two keypoints $\vec{\boldsymbol{p}}_1$ and $\vec{\boldsymbol{p}}_2$. Such sampled points will take the form $\vec{\boldsymbol{q}} = (1-t)\vec{\boldsymbol{p}}_1 + t\vec{\boldsymbol{p}}_2$, where t is a parameter within the $[0, 1]$ range. For additional robustness, an error term $\vec{\boldsymbol{\varepsilon}}$ is added to the sampled point $\vec{\boldsymbol{q}}$. This prevents us from misdetecting edges whose endpoints lie at the border of the edge (as all edges in this case have thickness, therefore it matters where the keypoints are located). The number of sampled points, T, can be taken in proportion with the length of the segment $\boldsymbol{p}_{12} = (\vec{\boldsymbol{p}}_1, \vec{\boldsymbol{p}}_2)$.

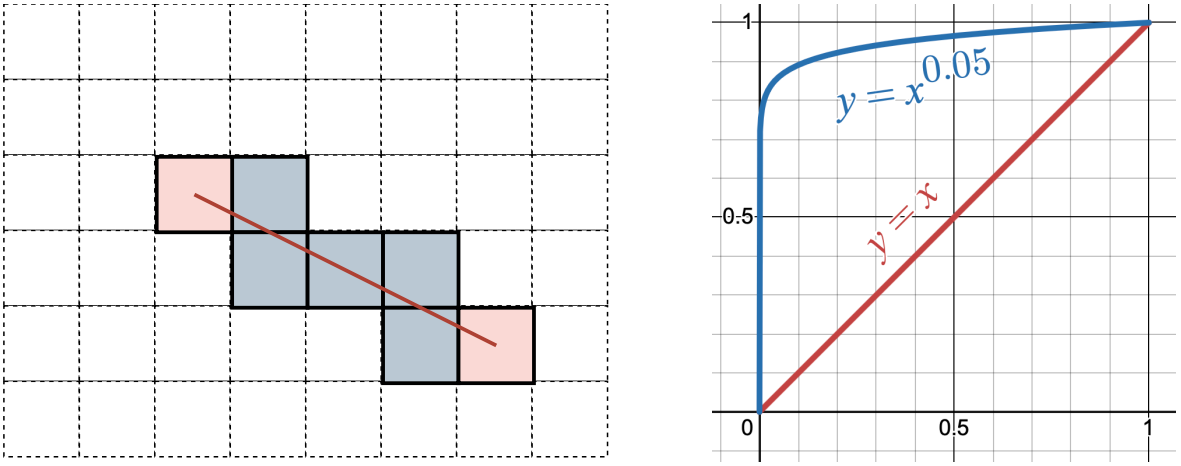


Figure 6: Sampling method for edge detection (left); Gamma corrections amplify pixels with small positive values (right)

Edge detection via sampling.

- *Input.*
  - Edge mask I_E of size $H \times W$;
  - Two keypoints $\vec{\boldsymbol{p}}_1$ and $\vec{\boldsymbol{p}}_2$;
  - $\gamma \approx 0.01$
- *Output.* Confidence if there is an edge connecting $\vec{\boldsymbol{p}}_1$ and $\vec{\boldsymbol{p}}_2$, between 0 and 1.

Step 1. Let $T := \lceil \|\vec{\boldsymbol{p}}_1 - \vec{\boldsymbol{p}}_2\| \rceil$.

Step 2. Generate T random values of $t^{(i)} \in [0, 1]$.

Step 3. For each $t^{(i)}$, the sampled point is $\vec{q}^{(i)} := \left(1 - t^{(i)}\right)\vec{p}_1 + t^{(i)}\vec{p}_2 + \vec{\varepsilon}$, where $\vec{\varepsilon}$ is a random noise vector, taking range $[-2, 2]$ for each dimension.

Step 4. Return confidence as the sum of the points' intensities raised to the power of γ:

$$\text{Confidence} := \sum_i I_E\left(q_x^{(i)}, q_y^{(i)}\right)^\gamma$$

3.3. Detecting groups of orthogonal lines via clustering of gradients

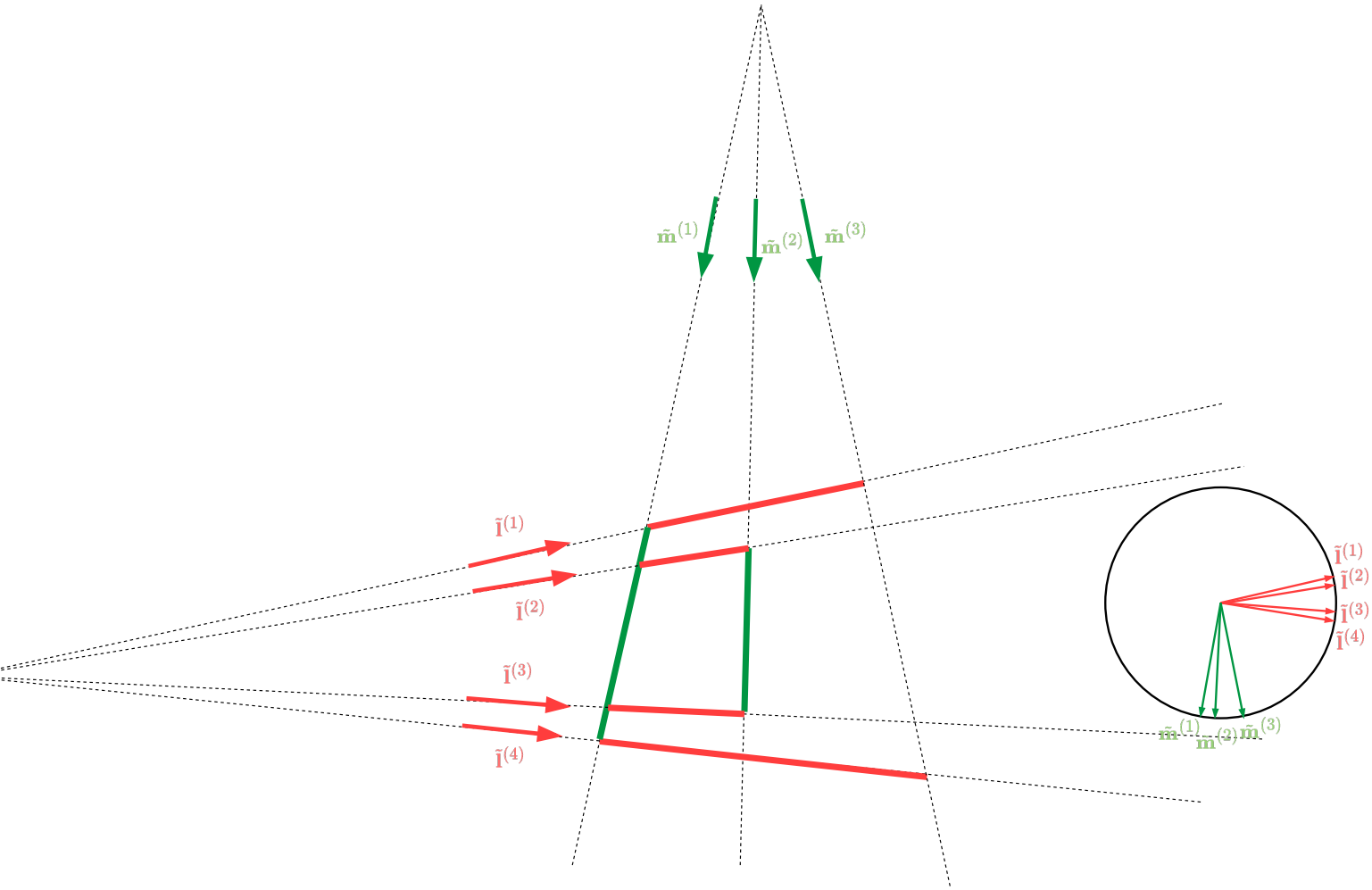


Figure 7: Classified segments based on the gradient vector
(red for vertical field lines, green for horizontal field lines, and yellow for upright)

Grouping lines by the gradient vector.

Input.

- Set of lines $L = \left\{\tilde{l}^{(i)}\right\}$

Output. Two sets of parallel lines L_1 and L_2, where each line in L_1 is orthogonal with a line in L_2 in the original space.

Step 1. For each line $\tilde{l}^{(i)}$, compute the normalised gradient vector $\hat{v}^{(i)} = \dfrac{1}{\sqrt{l_1^{(i)^2} + l_2^{(i)^2}}} \begin{bmatrix} l_1^{(i)} \\ l_2^{(i)} \end{bmatrix}$

Step 2. Use a clustering algorithm $\mathcal{C}$ to group the lines into two groups.

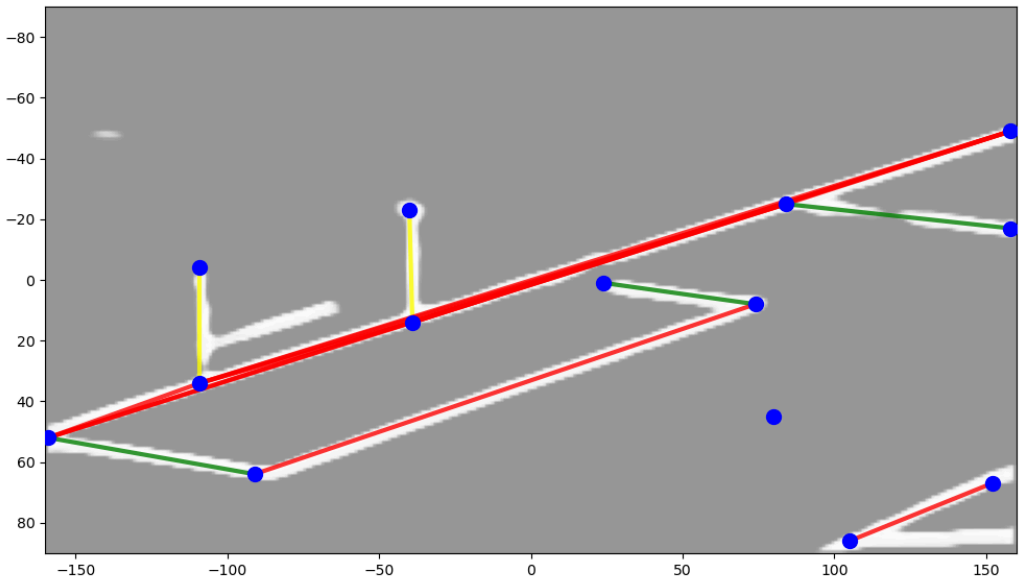


Figure 8: Classified segments based on the gradient vector
(red for vertical field lines, green for horizontal field lines, and yellow for upright)

4. Homographic rectification

For theoretical and computational methods regarding the homographic transformation, the report refers to Hartley and Zisserman [8, Chapter 2] on *Projective geometry and Transformations of 2D.*

4.1. Projective geometry

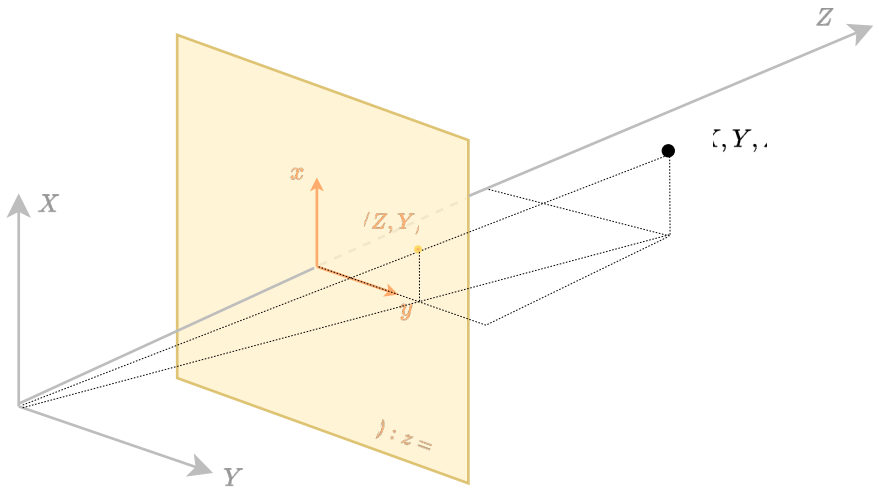


Figure 9: The homogenous coordinates system inspired by projective geometry, where a point in the world space (X, Y, Z) is mapped to the point $(X/Z, Y/Z)$ in the projective plane $z = 1$.

Consider the $\mathbb{R}^3$ space with the plane $(\pi) : z = 1$, which we denote as the **projective plane**. Let $\mathcal{H}$ be a set of points in $\mathbb{R}^3$ (a single point, a line, a shape, etc.) in $\mathbb{R}^3$. Then, for each point $\tilde{\boldsymbol{p}}$ in $\mathcal{H}$, the line through $\tilde{\boldsymbol{p}}$ and the origin intersects the projective plane (π) at a point denoted $\vec{\boldsymbol{p}}$. *We call* $\vec{\boldsymbol{p}}$ *the projective image of* $\tilde{\boldsymbol{p}}$ *onto the projective plane.*

This projection motivates us to derive an alternative representation for any point $\vec{\boldsymbol{p}} = [x \;\; y]^T$ on a 2-dimensional (2D) plane by considering it the projective image onto the plane $z = 1$ of any point $\tilde{\boldsymbol{p}}$ in $\mathbb{R}^3$. The coordinates of point $\vec{\boldsymbol{p}}$ in $\mathbb{R}^3$ is $[x \;\; y \;\; 1]^T$ as $\vec{\boldsymbol{p}}$ lies on the $z = 1$ plane. Using the argument of similar triangles, one can derive that all possible representations of the correspondent finite point $\tilde{\boldsymbol{p}}$ is thus $k[x \;\; y \;\; 1]^T = [kx \;\; ky \;\; k]^T$ for any $k \neq 0$.

Following this representation, any line $ax + by + c = 0$ on the $\mathbb{R}^2$ plane can also be uniquely defined by a single homogenous vector $\tilde{\boldsymbol{l}} = [a \;\; b \;\; c]^T$. This gives us the following important, yet computationally neat, corollaries:

- A point $\tilde{\boldsymbol{p}}$ is incident with (or lies on) a line $\tilde{\boldsymbol{l}}$ iff $\tilde{\boldsymbol{l}}^T\tilde{\boldsymbol{p}} = 0$.
- Two lines $\tilde{\boldsymbol{l}}_1$ and $\tilde{\boldsymbol{l}}_2$ always intersects at point $\tilde{\boldsymbol{p}} = \tilde{\boldsymbol{l}}_1 \times \tilde{\boldsymbol{l}}_2$ (whether the intersection actually corresponds to a real point on the 2D plane is a different problem).
- The line crossing two points $\tilde{\boldsymbol{p}}_1$ and $\tilde{\boldsymbol{p}}_2$ is computed as $\tilde{\boldsymbol{l}} = \tilde{\boldsymbol{p}}_1 \times \tilde{\boldsymbol{p}}_2$.

Note that the second corollary also works for parallel lines, that is, the two lines of form $\tilde{\boldsymbol{l}}_1 = [a \;\; b \;\; c_1]^T$ and $\tilde{\boldsymbol{l}}_2 = [a \;\; b \;\; c_2]^T$. The intersection of which is,

$$\tilde{\boldsymbol{l}}_1 \times \tilde{\boldsymbol{l}}_2 = (c_2 - c_1) \begin{bmatrix} b \\ a \\ 0 \end{bmatrix}$$

Thus, a homogenous vector whose last component is zero can be thought of as the "intersection" of parralel lines, representing "the point at infinity."

4.2. Transformations in 2D

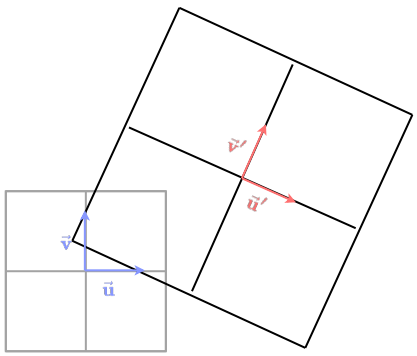
A geometric transformation T is a mapping from a shape $\mathcal{H}$ to its image $\mathcal{H}'$, preserving certain geometric identities between the original and the transformed shape.

We consider linear transformations in the homogenous coordinates system $\mathbb{P}^2$. These transformations take the form

$$\begin{aligned} H : \mathbb{P}^2 &\mapsto \mathbb{P}^2 \\ \tilde{\boldsymbol{p}} &\mapsto H\tilde{\boldsymbol{p}} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \end{aligned}$$

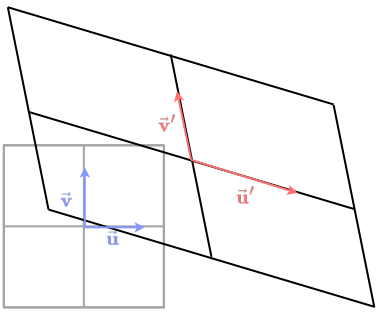
We consider the following 3 families of transformation:

- **Similarity**. A similarity is a transformation that preserves angles and length ratios. It can be further decomposed as a chain of a θ-rotation about the origin $R(\theta)$, followed by a scale k and a translation along the vector $\vec{\boldsymbol{v}}$.



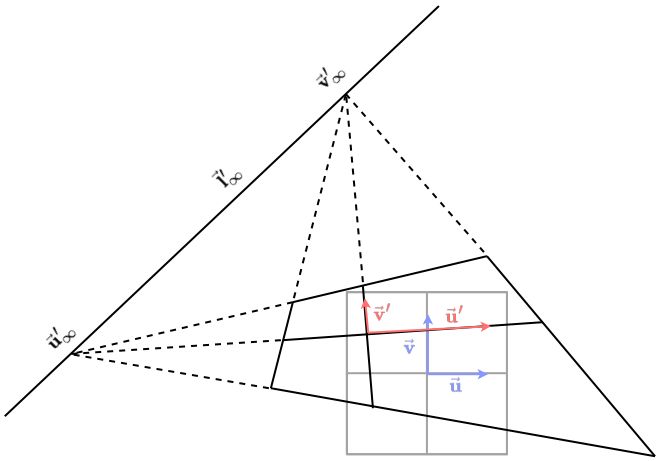
$$\tilde{\boldsymbol{p}}' = \begin{bmatrix} k\cos\theta & -k\sin\theta & v_x \\ k\sin\theta & k\cos\theta & v_y \\ 0 & 0 & 1 \end{bmatrix} \tilde{\boldsymbol{p}}$$

- **Affinity**. An affinity is a transformation that preserves parallelisms and length ratios. It can be described as a linear transformation in $\mathbb{R}^2$ where the basis $\{\hat{\boldsymbol{\imath}}, \hat{\boldsymbol{\jmath}}\}$ is transformed into $\{a_{11}\hat{\boldsymbol{\imath}} + a_{21}\hat{\boldsymbol{\jmath}}, a_{12}\hat{\boldsymbol{\imath}} + a_{22}\hat{\boldsymbol{\jmath}}\}$. A similarity is also a special case of an affinity where $a_{11} = a_{22} = k\cos\theta$ and $-a_{12} = a_{22} = k\sin\theta$.



$$\tilde{\boldsymbol{p}}' = \begin{bmatrix} a_{11} & a_{12} & v_x \\ a_{21} & a_{22} & v_y \\ 0 & 0 & 1 \end{bmatrix} \tilde{\boldsymbol{p}}$$

- **Homography**. A homography is a transformation that preserves incidence and collinearity, that is, if a point $\vec{\boldsymbol{p}}$ belongs to the line $\tilde{\boldsymbol{l}}$ (or $\tilde{\boldsymbol{p}}^T\tilde{\boldsymbol{l}} = 0$), then so is $\vec{\boldsymbol{p}'}$ on the transformed line $\tilde{\boldsymbol{l}}' = H^{-T}\tilde{\boldsymbol{l}}$ [8, p.33]. An affinity is a special case of a homography where $h_{31} = h_{32} = 0$.



$$\tilde{\boldsymbol{p}}' = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \tilde{\boldsymbol{p}}$$

In field registraion we are interested in recovering the homography matrix in two conditions:

- where all the points, pre-transformed $\left\{\tilde{\boldsymbol{p}}^{(i)}\right\}$ and post-transformed $\left\{\tilde{\boldsymbol{q}}^{(i)}\right\}$, and their correspondences $\tilde{\boldsymbol{p}}^{(i)} \leftrightarrow \tilde{\boldsymbol{q}}^{(i)}$ are well established *(The DLT algorithm)*; or
- where the pre-transformed $P = \left\{\tilde{\boldsymbol{p}}^{(i)}\right\}$ and post-transformed $Q = \left\{\tilde{\boldsymbol{q}}^{(i)}\right\}$ points are not fully known, and no correspondences have been found. *(Rectification + RANSAC)*.

4.3. Solving linear systems using the Singular Value Decomposition (SVD)

Often when solving for parameters of a certain geometric entity or transformation, it is needed to solve for the non-trivial roots of the correspondent linear system, for instance, $A\vec{\boldsymbol{x}} = \vec{\boldsymbol{0}}$. However, due to noise and errors arising from preceding calculations, the system can either be underdetermined or overdetermined.

A common workaround is to frame the problem as a minimisation of the norm of the residual vector $\|A\hat{\boldsymbol{x}}\|^2$, with the restriction $\|\hat{\boldsymbol{x}}\|^2 = 1$ to avoid the trivial solution $\hat{\boldsymbol{x}} = \boldsymbol{0}$. One common method for these types of problem is the Singular Value Decomposition (SVD) algorithm. From the SVD, any matrix $A_{m\times n}$ can be written as

$$A = UDV^T = \sum_{i=1}^{n} \sigma_i \hat{\boldsymbol{u}}_i \hat{\boldsymbol{v}}_i^T$$

where $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ are the singular values of A, and $U = [\hat{\boldsymbol{u}}_1 \;\; \hat{\boldsymbol{u}}_2 \;\; \cdots \;\; \hat{\boldsymbol{u}}_r]$, and $V = [\hat{\boldsymbol{v}}_1 \;\; \hat{\boldsymbol{v}}_2 \;\; \cdots \;\; \hat{\boldsymbol{v}}_r]$ satisfying $U^T U = V^T V = I$.

This representation gives us a straightforward solution: $\arg \min_{\hat{\boldsymbol{x}}:\, \|\hat{\boldsymbol{x}}\|=1} \|A\hat{\boldsymbol{x}}\|^2 = \hat{\boldsymbol{v}}_r$, i.e. the right vector associated with the least non-zero singular value. A short proof is given in [8, A5.3]. The SVD sets the foundations for solving linear systems describing homographic transformations under imperfect conditions.

4.4. Recovering homography using point-to-point correspondence (Direct Linear Transform, DLT)

Given k point-to-point correspondences $\vec{\boldsymbol{p}}^{(i)} \leftrightarrow \vec{\boldsymbol{q}}^{(i)}$, where $k \geq 4$, find a homographic transformation H that maps every point $\vec{\boldsymbol{p}}^{(i)}$ to $\vec{\boldsymbol{q}}^{(i)}$. Note that each correspondence $\vec{\boldsymbol{p}}^{(i)} \leftrightarrow \vec{\boldsymbol{q}}^{(i)}$ provides the equation $H\tilde{\boldsymbol{p}}^{(i)} \sim \tilde{\boldsymbol{q}}^{(i)}$, or:

$$
\begin{aligned}
& H\tilde{\boldsymbol{p}}^{(i)} \times \tilde{\boldsymbol{q}}^{(i)} = \tilde{\mathbf{0}} \\
\Leftrightarrow & \begin{cases}
\left(h_{11}p_1^{(i)} + h_{12}p_2^{(i)} + h_{13}p_3^{(i)}\right)q_2^{(i)} - \left(h_{21}p_1^{(i)} + h_{22}p_2^{(i)} + h_{23}p_3^{(i)}\right)q_1^{(i)} = 0 \\
\left(h_{21}p_1^{(i)} + h_{22}p_2^{(i)} + h_{23}p_3^{(i)}\right)q_3^{(i)} - \left(h_{31}p_1^{(i)} + h_{32}p_2^{(i)} + h_{33}p_3^{(i)}\right)q_2^{(i)} = 0 \\
\left(h_{31}p_1^{(i)} + h_{32}p_2^{(i)} + h_{33}p_3^{(i)}\right)q_1^{(i)} - \left(h_{11}p_1^{(i)} + h_{12}p_2^{(i)} + h_{13}p_3^{(i)}\right)q_3^{(i)} = 0
\end{cases} \\
\Leftrightarrow & \begin{bmatrix}
p_1^{(i)}q_2^{(i)} & p_2^{(i)}q_2^{(i)} & p_3^{(i)}q_2^{(i)} & -p_1^{(i)}q_1^{(i)} & -p_2^{(i)}q_1^{(i)} & -p_3^{(i)}q_1^{(i)} & & & \\
 & & & p_1^{(i)}q_3^{(i)} & p_2^{(i)}q_3^{(i)} & p_3^{(i)}q_3^{(i)} & -p_1^{(i)}q_2^{(i)} & -p_2^{(i)}q_2^{(i)} & -p_3^{(i)}q_2^{(i)} \\
-p_1^{(i)}q_3^{(i)} & -p_2^{(i)}q_3^{(i)} & -p_3^{(i)}q_3^{(i)} & & & & p_1^{(i)}q_1^{(i)} & p_2^{(i)}q_1^{(i)} & p_3^{(i)}q_1^{(i)}
\end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ \vdots \\ h_{31} \\ h_{32} \\ h_{33} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \\
\Leftrightarrow & A^{(i)}\tilde{\boldsymbol{h}} = \tilde{\mathbf{0}}
\end{aligned}
$$

Note that the left matrix $A^{(i)}$ has rank 2, therefore we only need to keep two of the three rows. As a homographic transformation H has 8 DoF, a minimum of 4 correspondences is required for the system to be determined.

The Direct Linear Transform (DLT) algorithm.

Input. k correspondences $p^{(i)} \mapsto q^{(i)}$ under the homographic transformation H.
Output. Best-fitting homography matrix H.

Step 1. For each correspondence $\tilde{\boldsymbol{p}}^{(i)} \leftrightarrow \tilde{\boldsymbol{q}}^{(i)}$, prepare the matrix

$$A^{(i)} = \begin{bmatrix} -p_1^{(i)}q_3^{(i)} & -p_2^{(i)}q_3^{(i)} & -p_3^{(i)}q_3^{(i)} & & & & p_1^{(i)}q_1^{(i)} & p_2^{(i)}q_1^{(i)} & p_3^{(i)}q_1^{(i)} \\ & & & p_1^{(i)}q_3^{(i)} & p_2^{(i)}q_3^{(i)} & p_3^{(i)}q_3^{(i)} & -p_1^{(i)}q_2^{(i)} & -p_2^{(i)}q_2^{(i)} & -p_3^{(i)}q_2^{(i)} \end{bmatrix}$$

Step 2. Stack all the matrices $A^{(i)}$ into a single matrix A of size $2k \times 9$:

$$A = \begin{bmatrix} A^{(1)} \\ A^{(2)} \\ \vdots \\ A^{(k)} \end{bmatrix}$$

Step 3. Solve for $\tilde{\boldsymbol{h}} = \arg\min_{\|\tilde{\boldsymbol{h}}'\|=1} \|A\tilde{\boldsymbol{h}}'\|^2$ via the SVD. *If $k = 4$, the linear system is consistent.*

Step 4. Reshape $\tilde{\boldsymbol{h}}$ from size 9×1 to the matrix $H \in \mathbb{R}^{3\times3}$.

4.5. Rectification

Given two groups of parallel lines $L = \begin{bmatrix} \cdots & \tilde{\boldsymbol{l}}^{(i)} & \cdots \end{bmatrix}$ and $M = \begin{bmatrix} \cdots & \widetilde{\boldsymbol{m}}^{(j)} & \cdots \end{bmatrix}$ such that each pair of lines $\left(\tilde{\boldsymbol{l}}^{(i)}, \widetilde{\boldsymbol{m}}^{(j)}\right)$ is orthogonal. Under the homography H, the groups become $L' = HL = \begin{bmatrix} \cdots & \tilde{\boldsymbol{l}}'^{(i)} & \cdots \end{bmatrix}$ and $M' = HM = \begin{bmatrix} \cdots & \widetilde{\boldsymbol{m}}'^{(j)} & \cdots \end{bmatrix}$. The task here is to recover the homography H.

We can approach this problem step-by-step, implying each additional constraint at each step:

- Firstly, to restore affinity, that is, to apply a "pure" homography

$$H_P = \begin{bmatrix} 1 & & \\ & 1 & \\ v_x & v_y & 1 \end{bmatrix}$$

  such that if $L' \overset{H_P}{\mapsto} L_P$ and $M \overset{H_P}{\mapsto} M_P$, then each line in L_P is pairwise parallel, and each line in M_P is pairwise parallel. The process is called **affine rectification**. [8, p.49-52, Section 2.7.2].

- Secondly, to restore similarity, that is, to apply a "pure" affinity

$$H_A = \begin{bmatrix} a_{11} & a_{12} & \\ a_{21} & a_{22} & \\ & & 1 \end{bmatrix}$$

  such that if $L_P \overset{H_A}{\mapsto} L_A$ and $M_P \overset{H_P}{\mapsto} M_A$, then the transformed image is only different from the original one by a similarity. This process is called **metric recrification** [8, p.55-58, Section 2.7.5].

- Finally, to restore the original coordinates of the lines, that is, to apply a similarity

$$H_S = \begin{bmatrix} k\cos\theta & -k\sin\theta & t_x \\ k\sin\theta & k\cos\theta & t_y \\ & & 1 \end{bmatrix}$$

such that if $L_A \overset{H_S}{\mapsto} L_S$ and $M_A \overset{H_P}{\mapsto} M_S$, then $L_S = L$ and $M_S = M$, that is the original lines are fully recovered.

The rectification homography is thus $H' = H_P H_A H_S$, therefore the homography from the original image to the transformed image is the inverse, $H = H'^{-1} = H_S^{-1} H_A^{-1} H_P^{-1}$.

4.5.1. Similarity rectification

The task at this stage is to find a similarity H_S to map a set of keypoints and segments $\boldsymbol{P}$ and the original field $\boldsymbol{Q}$, with the notice that

- Only a small subset of keypoints and segments in $\boldsymbol{Q}$ has a correspondence in $\boldsymbol{P}$;
- No direct correspondence between segments or keypoints are known; and
- A segment $\boldsymbol{p}^{(ij)} := \left(\vec{\boldsymbol{p}}^{(i)}, \vec{\boldsymbol{p}}^{(j)}\right)$ may only map to a portion of the segment $\boldsymbol{q}^{(ij)} := \left(\vec{\boldsymbol{q}}^{(i)}, \vec{\boldsymbol{q}}^{(j)}\right)$. Nonetheless, there is a high chance that there exists a segment in $\boldsymbol{P}$ that perfectly maps onto $\boldsymbol{Q}$.

Moreover, we notice that a similarity has 4 degrees of freedom. Indeed, let $(a, b, c, d) := \left(k\cos\theta, k\sin\theta, t_x, t_y\right)$, the similarity takes the form $H_S = \begin{bmatrix} a & -b & c \\ b & a & d \\ & & 1 \end{bmatrix}$. Therefore, it is sufficient to indicate a segment at pre-transformation $\left(\vec{\boldsymbol{p}}^{(i)}, \vec{\boldsymbol{p}}^{(j)}\right)$ and post-transformation $\left(\vec{\boldsymbol{q}}^{(i)}, \vec{\boldsymbol{q}}^{(j)}\right)$.

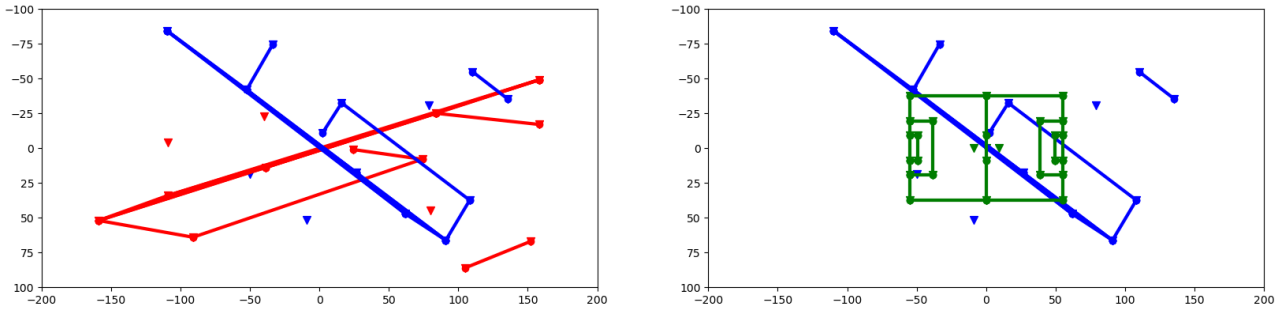


Figure 10: Transformation $H_P H_A$ from original segments (red) to metrically rectified segments (blue) (left); Metrically rectified segments (blue) and the target field (green)

Similarity rectification from a correspondence of two segments.

- *Input*: Four points $\vec{\boldsymbol{p}}^{(1)}, \vec{\boldsymbol{p}}^{(2)}, \vec{\boldsymbol{q}}^{(1)}, \vec{\boldsymbol{q}}^{(2)}$ indicating two segments $\vec{\boldsymbol{p}}^{(1)}, \vec{\boldsymbol{p}}^{(2)}$ and $\vec{\boldsymbol{q}}^{(1)}, \vec{\boldsymbol{q}}^{(2)}$.
- *Output*: The similarity H_S.

Step 1. Let $\left(p_1^{(i)}, p_2^{(i)}, 1\right) := \tilde{\boldsymbol{p}}^{(i)}$, $\left(q_1^{(i)}, q_2^{(i)}, 1\right) := \tilde{\boldsymbol{q}}^{(i)}$ for $i = 1, 2$.

Step 2. Solve the linear system $H_S \tilde{\boldsymbol{p}}^{(i)} = \tilde{\boldsymbol{q}}^{(i)}$.

$$\begin{cases} ap_1^{(1)} - bp_2^{(1)} + c = q_1^{(1)} \\ ap_2^{(1)} + bp_1^{(1)} + d = q_2^{(1)} \\ ap_1^{(2)} - bp_2^{(2)} + c = q_1^{(2)} \\ ap_2^{(2)} + bp_1^{(2)} + d = q_2^{(2)} \end{cases} \quad \therefore \quad \begin{bmatrix} p_1^{(1)} & -p_2^{(1)} & 1 & 0 \\ p_2^{(1)} & p_1^{(1)} & 0 & 1 \\ p_1^{(2)} & -p_2^{(2)} & 1 & 0 \\ p_2^{(2)} & p_1^{(2)} & 0 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} q_1^{(1)} \\ q_2^{(1)} \\ q_1^{(2)} \\ q_2^{(2)} \end{bmatrix}$$

The similarity can thus be solved using any linear solver such as the SVD.

The final task is to actually find such a correspondence of segments. With non-labelled segments, the only method is to bruteforce through each pair of segments. The question is how to derive a metric to evaluate if a correspondence is "appropriate."

We borrow some ideas from the "RANdom SAmple Consensus" (RANSAC) [8, p.117, Section 4.7.1] to propose an evaluation metric of a segment correspondence, using the pairwise "count of inliers" metric. Suppose after the similarity transformation we find the correspondences $\boldsymbol{p}^{(a_i b_i)} \leftrightarrow \boldsymbol{q}^{(c_i, d_i)}$. Then we want to the maximise the number of segments in P having a Q-correspondence, and vice versa. Formally, denote S_P be the set of segments in P whose Q-correspondence exists, and similarity denote S_Q. Then the quantity $|S_P| + |S_Q|$ represents the "inliers" and is desired to be maximised.

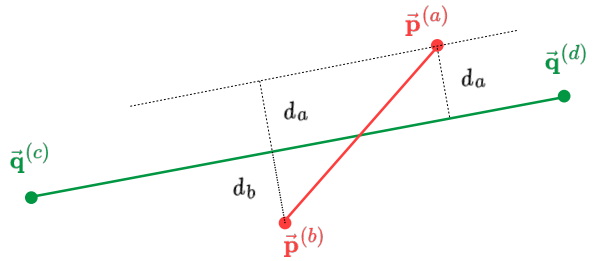


Figure 11: A segment $\boldsymbol{p}^{a_i b_i}$ is considered as "inlier" with respect to the segment $\boldsymbol{q}^{c_j d_j}$ if $\operatorname{dist}(\boldsymbol{p}^a, \boldsymbol{q}^{cd}) + \operatorname{dist}(\boldsymbol{p}^b, \boldsymbol{q}^{cd})$ is less than a certain threshold

Modified RANSAC to find segment correspondence.

Step 1. Let $S_P := \emptyset$ and $S_Q := \emptyset$

Step 2. Select a random segment $\boldsymbol{p}^{(\alpha\beta)} = \left(\vec{\boldsymbol{p}}^{(\alpha)}, \vec{\boldsymbol{p}}^{(\beta)}\right)$ in $\boldsymbol{P}$ and a random segment $\boldsymbol{q}^{(\gamma\delta)} = \left(\vec{\boldsymbol{q}}^{(\gamma)}, \vec{\boldsymbol{q}}^{(\delta)}\right)$.

Step 3. Find a similarity H_S such that the segment $\boldsymbol{p}^{(\alpha\beta)}$ is mapped to $\boldsymbol{q}^{(\gamma\delta)}$.

Step 4. For each segment $\boldsymbol{p}^{(ab)}$ in $\boldsymbol{P}$ and $\boldsymbol{q}^{(cd)}$

- Calculate the distance between each point in segment $\boldsymbol{p}^{(ab)}$ to the line containing $\boldsymbol{q}^{(cd)}$:

$$d_a := \mathrm{dist}\left(\vec{\boldsymbol{p}}^{(a)}, \boldsymbol{q}^{(cd)}\right)$$
$$d_b := \mathrm{dist}\left(\vec{\boldsymbol{p}}^{(b)}, \boldsymbol{q}^{(cd)}\right)$$

- Let $d_{\mathrm{avg}} = \frac{1}{2}(d_a + d_b)$. If $d_{\mathrm{avg}} \leq T$, $S_P := S_P \cup \{ab\}$, $S_Q := S_Q \cup \{cd\}$.

Step 5. Let $\mathrm{Score} = |S_P| + |S_Q|$. If $\mathrm{Score} \geq \mathrm{Score}_{\mathrm{Accept}}$, return H_S. Otherwise, repeat the process.

5. Experiments

5.1. Dataset Preparation

The SoccerNet Calibration Dataset (`sn_calibration`) [1] consists of 12356 training samples, 2796 validation samples and 2719 testing samples. Our actual training and validation processes only use a subset of the given samples for model prototyping purposes.

Task	Training	Validation	Testing
General	12356	2796	2719
Edge segmentation	First 128	First 64	First 64
Keypoint concentration	First 128	First 128	First 128

Table 1:

Each sample contains two files: a `PNG` colour image of size 960×540px and a `JSON` data file containing the coordinates of 26 different field lines and curves. For this project, we ignore the three curves (the 10-yard kick-off circle and the two semicircles). Each of the lines is described by a segment (or a poly-segment) of two or more keypoints.

Assuming every football field has the same size and layout[3] of 110×75m, we can always map a template football field to the actual field displayed on the image via a homography H. Such homography can be reconstructed using the DLT algorithm when more than 4 point-to-point correspondences are found.

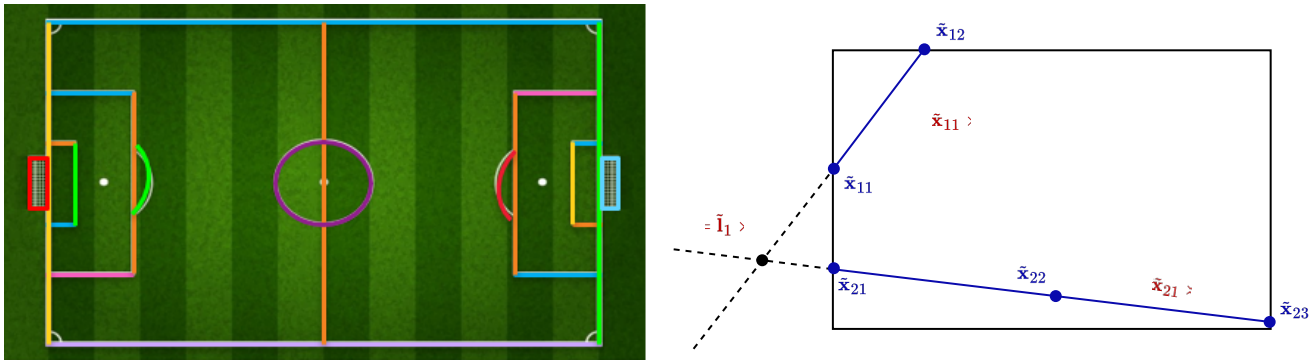


Figure 12: Defined field lines in the dataset [1] (left); Keypoint recovery method (right)

For instance, given the coordinates of the field's top and left border lines, the top left corner keypoint can be computed as:

$$\tilde{p}_{\text{FIELD\_TOP\_LEFT}} = \tilde{l}_{\text{FIELD\_BOX\_TOP}} \times \tilde{l}_{\text{FIELD\_BOX\_LEFT}}$$

This can be done for all the visible keypoints on the field, as well as off-screen keypoints whose constituent lines are visible. Using the found correspondences, apply the DLT algorithm directly

[3]This is not always the case. The current regulation allows for fields to have a variable size of 90-120m in length and 50-90m in width, therefore each field may have a different size than another. Our project uses the field size of 110×75m.

to find the homography H. All other keypoints can be found by applying the homography on the template field. Given that at least 4 non-collinear keypoints are positioned, the coordinates of the other of the 27 keypoints can be detected via a homography transformation. This can be done via the DLT algorithm.

The dataset is then catered to different types of problem via `pytorch`'s `Datasets`. For efficient batching, the labels must be in a `Tensor` form for stacking and parallel computation. Therefore, we implemented a `Dataset` interface for every use case in the pipeline, each reading from the same original dataset from SoccerNet [1], but with additional preprocessing (such as downscaling to size $H \times W$, grayscaling, homography reconstruction and transformation, etc.).

Task / `Dataset` name	**Input**	**Output / Labels**
General `CalibrationDataset`	Raw image $(3 \times H \times W)$	Raw `JSON` dictionary
Edge segmentation `MaskingCalibrationDataset`	Grayscaled image $(H \times W)$	Image mask, highlighted field lines $(H \times W)$

Keypoint concentration
`MaskingCalibrationDataset`
`(keypoint_only=True)`
	Grayscaled image
	$(H \times W)$
		Image mask, highlighted keypoints
		$(H \times W)$

Keypoints `PitchKeypointCalibrationDataset`	Grayscaled image $(H \times W)$	List of all keypoints, reconstructed via homography (27×3)
Homography reconstruction `PitchHomographyCalibrationDataset`	Grayscaled image $(H \times W)$	The homography matrix (3×3)

Table 2: Custom `Datasets` for different tasks. $H = 180, W = 320$

5.2. Image segmentation

Two U-Net models $\mathcal{U}_{\text{seg}}$ and $\mathcal{U}_{\text{kp}}$ are trained based on the procedure stated in Section 2.2, for $n_{\text{seg}} = 64$ epochs and $n_{\text{kp}} = 48$ epochs respectively. From the training and validation losses, we observe that:

- On one hand, the $\mathcal{U}_{\text{seg}}$ model performs well with the segmentation task, with both the training loss and validation loss decreasing gradually. A small bump in losses is observed at $n = 62$, but it is suspected that for larger batches, the errors will decrease again. Qualitatively, the model is able to extract almost every pixels constituting the field lines. A small set of failed examples are probably caused by irregular recording angles or constrast values.
- On the other hand, the $\mathcal{U}_{\text{kp}}$ model suffers from overfitting: throughout 48 epochs, the validating loss is consistently higher than the training loss, despite an overall downward trend. Qualitative results show that despite being able to output an image mask consisting of points with thickness, some keypoints are not the intersection of any two field lines.

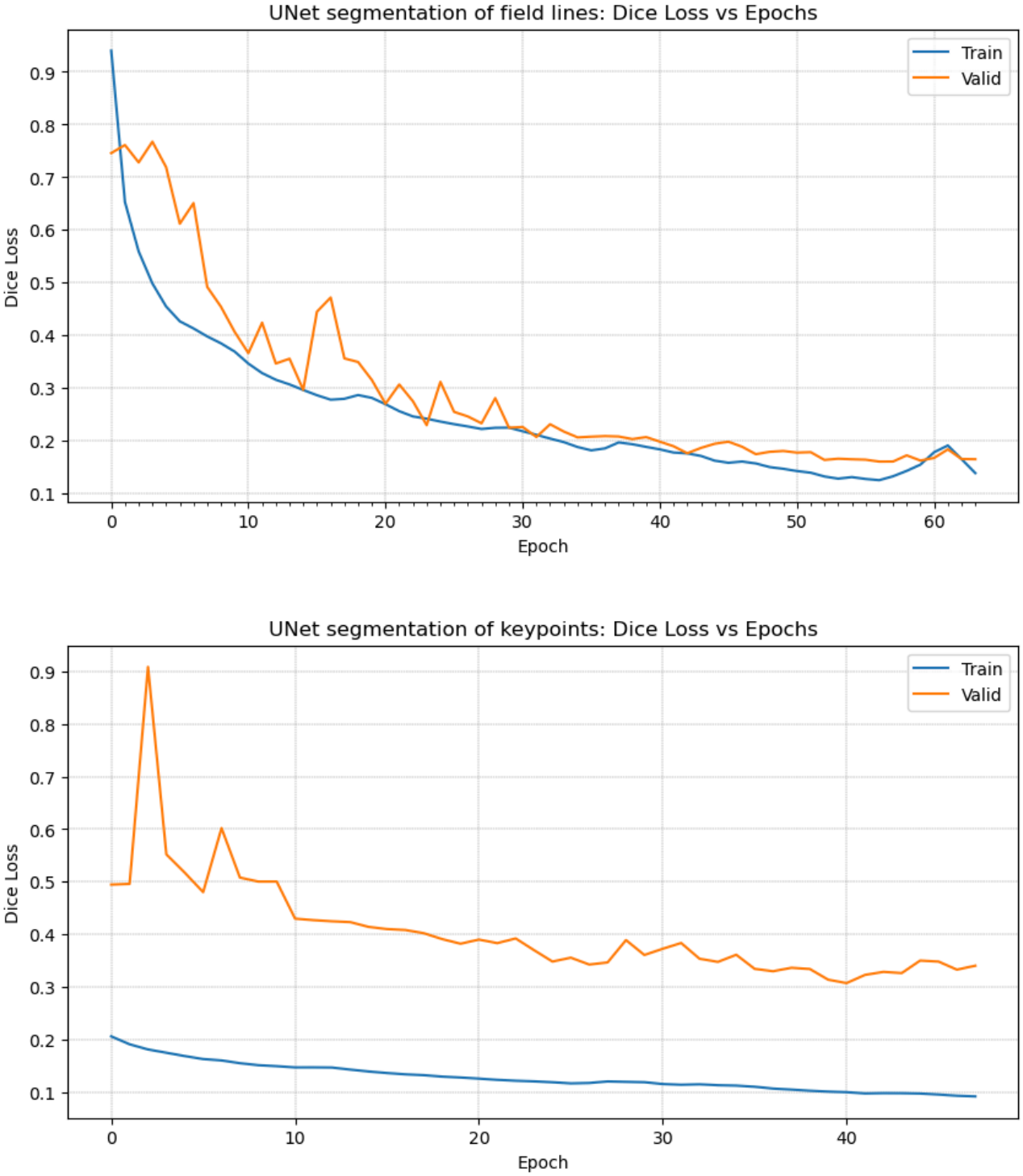


Figure 13: Training and Validation metrics for training $\mathcal{U}_{\text{seg}}$ and $\mathcal{U}_{\text{kp}}$

5.3. Field registration via homography transformation

We run the full pipeline for 32 images in the validation dataset. Overall, the pipeline works best for panaromic views towards the goalkeeper's and the penalty box, as these sections on the field have sufficient orthogonal lines for homography rectification. However, in some certain conditions, the pipeline fails to detect the correct segments on the field, leading to unmeaningful results at the end of the pipeline, or cause the whole process to halt without results. The three figures below illustrate a qualitative evaluation of the results.

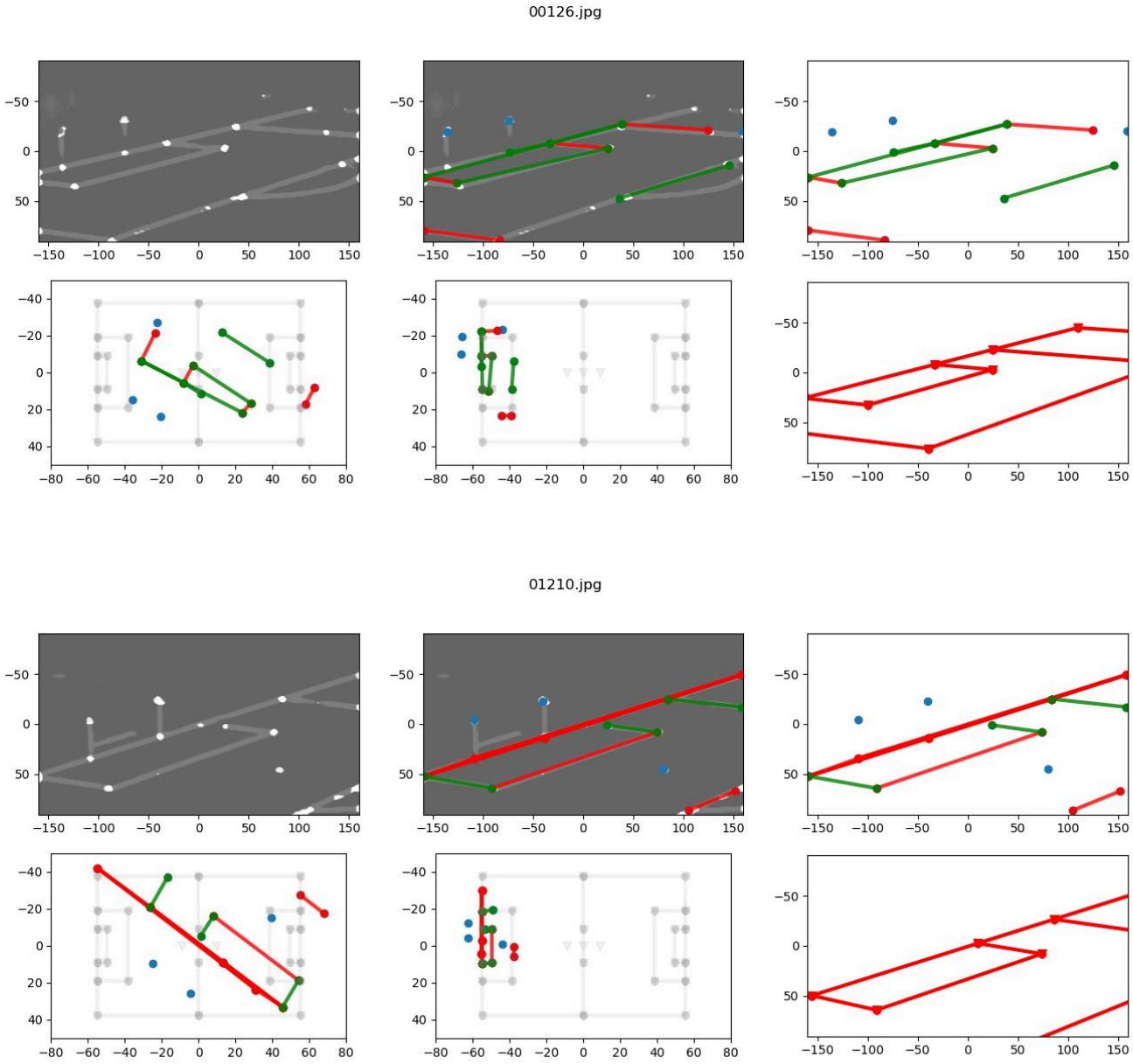


Figure 14: Correctly registered fields

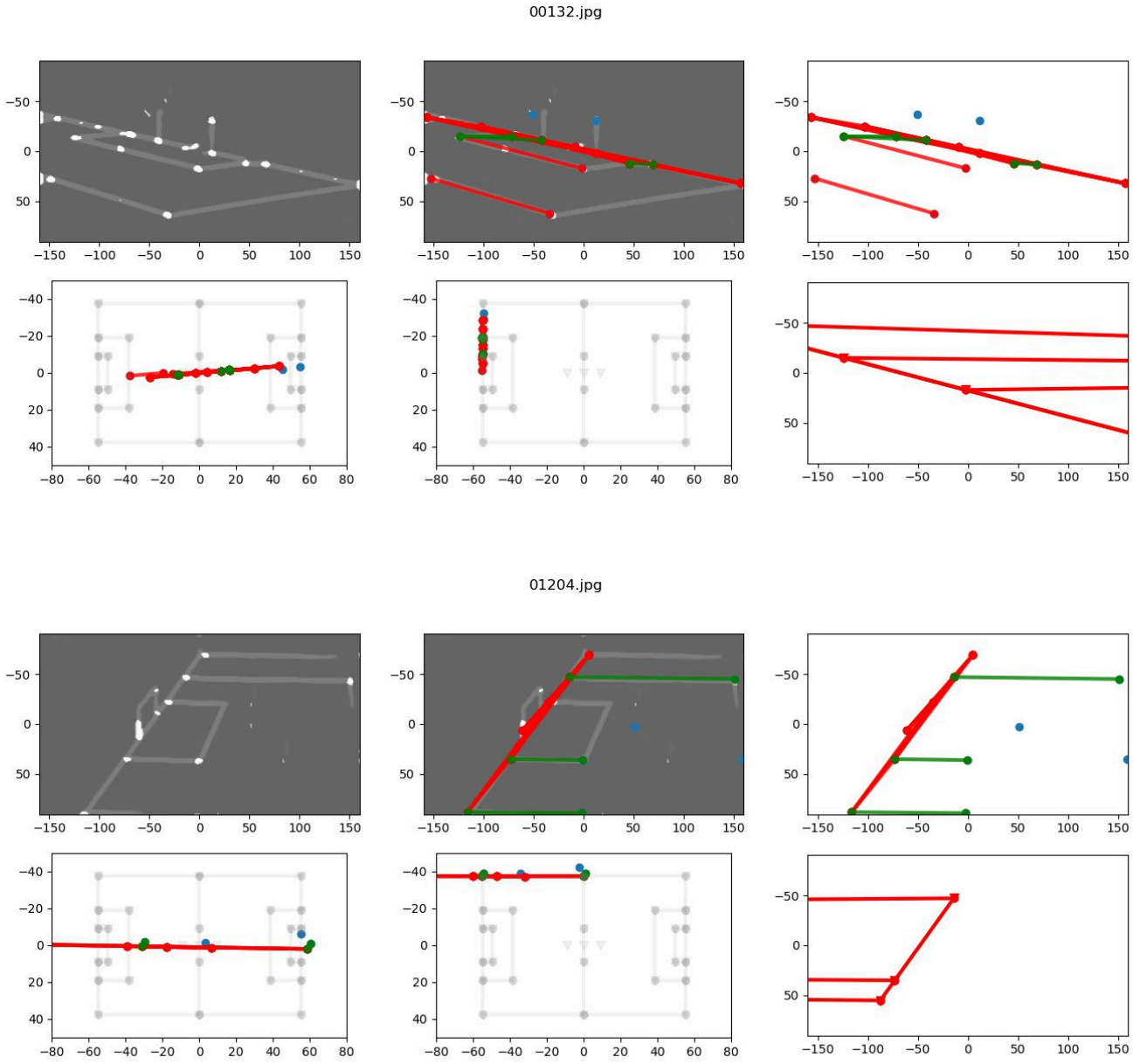


Figure 15: Registration failed due to insufficient keypoints and segments

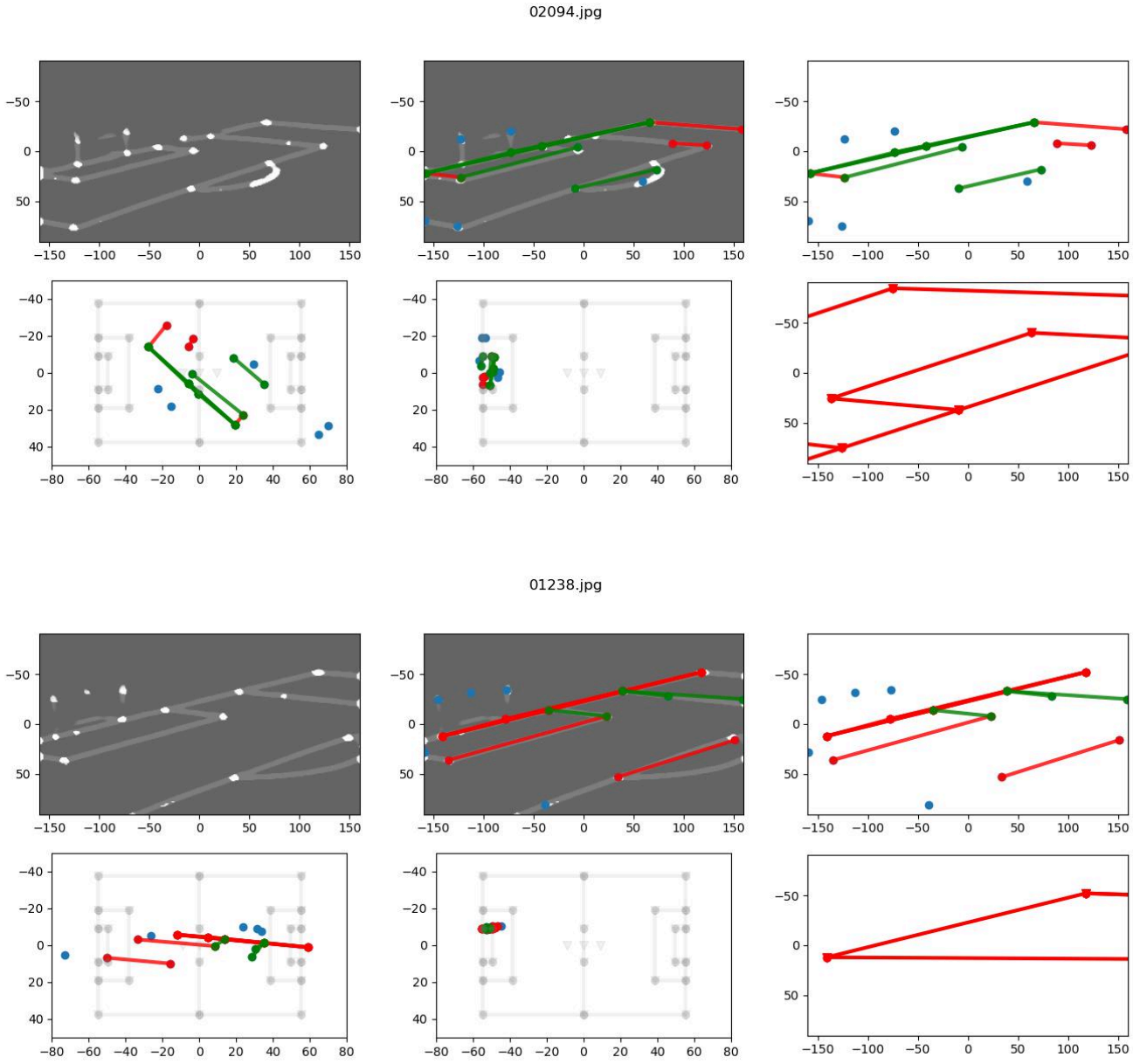


Figure 16: Registration failed due to mismatches between the criteria and the actual dataset

6. Discussion

6.1. Limitations and Mitigations

Feature masks generation. The choice of the Dice loss function seems to work fine for line extraction; however, it fails for keypoint detection. We aim to imporve this by exploring other augmentations to the loss function, such as regularisation techniques, or enforcing geometric invariants, as the field can be decomposed as a collection of geometrical objects (points, lines, polygons, conics, etc.). Moreover, additional data augmentation techniques should be applied to achieve more reliable results, especially before feeding to the segmentation model, such as contrast and brightness control, or classical feature extractions algorithms.

Graph creation. We found out in the validation process that in a large number of cases, the detected edges are insufficient for the homography recovering step. Therefore, more robust algorithms and techniques should be considered as well.

Homography reconstruction. Numerical stability is an issue for extreme cases (for e.g., when lines are almost in parallel). To mitigate this, we will consider different representation of the homography transform (such as the 4-point representation [12], camera displacement [13], etc.)

6.2. Future Plans

For the next stage of the project, we aim to mitigate the above limitations and finish the pipeline with the *Camera calibration* task via Zhang's [5] or Bradski's [6] algorithm. This result will then be verifiable through SoccerNet's [1] Camera Calibration task with its predefined metric. Finally, we will consider two possible pathways to extend the project:

- Expanding the pipeline for other sports field (basketball, hockey, or other field sports)
- Use the result of *Camera calibration* for other 3D reconstruction tasks, such as Human-pose estimation for players.

Bibliography

[1] "SoccerNet - 2025." Accessed: July 26, 2025. [Online]. Available: https://www.soccer-net.org/challenges/2025

[2] Inside FIFA, "Semi-automated offside technology to be used at FIFA World Cup 2022™." Accessed: July 26, 2025. [Online]. Available: https://inside.fifa.com/media-releases/semi-automated-offside-technology-to-be-used-at-fifa-world-cup-2022-tm

[3] ASEAN United FC, "🇵🇭 PHILIPPINES 2-1 THAILAND 🇹🇭 | #MITSUBISHIELECTRICCUP 2024 SEMI-FINAL LEG 1." Accessed: July 26, 2025. [Online]. Available: https://www.youtube.com/watch?v=mk0tK4X_KqQ

[4] ASEAN United FC, "🇹🇭 THAILAND 3-1 PHILIPPINES 🇵🇭 (4-3 AGG.) | #MITSUBISHIELECTRICCUP 2024 SEMI-FINAL LEG 2." Accessed: July 26, 2025. [Online]. Available: https://www.youtube.com/watch?v=aoC8EJ6QHuQ

[5] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Trans. Pattern Anal. Machine Intell.*, vol. 22, no. 11, pp. 1330–1334, 2000, doi: 10.1109/34.888718.

[6] G. Bradski, "ICRA 2010 OpenCV Tutorial." Accessed: July 26, 2025. [Online]. Available: https://wiki.ros.org/Events/ICRA2010Tutorial?action=AttachFile&do=view&target=ICRA_2010_OpenCV_Tutorial.pdf

[7] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation." Accessed: Nov. 03, 2025. [Online]. Available: http://arxiv.org/abs/1505.04597

[8] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2003.

[9] F. Neha, D. Bhati, D. K. Shukla, and M. Amiruzzaman, "From classical techniques to convolution-based models: A review of object detection algorithms." Accessed: Nov. 12, 2025. [Online]. Available: http://arxiv.org/abs/2412.05252

[10] "Loss Functions in the Era of Semantic Segmentation: A Survey and Outlook." Accessed: Nov. 08, 2025. [Online]. Available: https://arxiv.org/html/2312.05391

[11] I. Schlag and O. Arandjelovic, *Ancient Roman Coin Recognition in the Wild Using Deep Learning Based Recognition of Artistically Depicted Face Profiles*. 2017. doi: 10.1109/ICCVW.2017.342.

[12] D. DeTone, T. Malisiewicz, and A. Rabinovich, "Deep Image Homography Estimation." Accessed: Oct. 14, 2025. [Online]. Available: http://arxiv.org/abs/1606.03798

[13] "OpenCV: Basic concepts of the homography explained with code." Accessed: Nov. 13, 2025. [Online]. Available: https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html#tutorial_homography_Demo3

A. Project Github repository

https://github.com/SamNguyen2k5/NUSC-ISC-FootballAnalysis/tree/isc-checkpoint-1